Verificación de programas

Dante Zanarini

LCC

21 de noviembre de 2024

Qué es

- ¿Qué es un programa?
  ...
- ¿Qué es verificar?
  Según la RAE,
  Del lat. tardío verificāre.
  Comprobar o examinar la verdad de algo.
  Salir cierto y verdadero lo que se dijo o pronosticó.

Conceptos

- La especificación se puede pensar como una descripción formal de la tarea que un programa tiene que realizar.
- Responde a la pregunta ¿qué hace el programa?
- En este contexto, una especificación se puede pensar como un contrato entre el programador y el potencial usuario.
- El programa mismo, usualmente llamado *implementación*, responde a la pregunta ¿cómo se realiza la tarea?

Ejemplo

- El usuario propone al programador el problema de calcular la raíz cuadrada de un número real dado.

Preguntas

- ¿Para qué verificar software?
- ¿Vale la pena verificar software?
- ¿Qué me aporta?
- ¿Sirve más allá de lo académico?
- Si yo verifiqué un programa, ¿este nunca va a fallar?
- ...

Principales ingredientes para la verificación formal de programas

- Un lenguaje de programación;
- Una semántica para dicho lenguaje;
- Un lenguaje para especificar propiedades sobre programas;
- Un mecanismo para demostrar que un programa satisface una especificación;

Nuestro lenguaje de programación

- Como cualquier lenguaje de programación imperativo, distinguimos expresiones de comandos.
- Expresiones enteras:

```
E    ::=    n   | x | (-E)
            | (E + E) | (E * E) | (E - E)
```

donde $\mathtt{n} \in \mathbb{Z}$, $\mathtt{x} \in \mathit{Var}$

- Expresiones booleanas

```
B    ::=    true  | false | (!B)
            | (B & B) | (B || B) | (E < E)
```

- Convenciones sintácticas, abreviaturas, ...

Nuestro lenguaje de programación (2)

- Nuestro lenguaje de Comandos es:

```
C ::= x = E
    | C ; C
    | if B {C} else {C}
    | while B {C}
```

Semántica del lenguaje de programación

(intuitiva)

- El comando **atómico** $\mathtt{x} = \mathtt{E}$ se comporta como la asignación usual; evalúa la expresión entera $\mathtt{E}$ en el estado actual del almacenamiento y sobre-escribe el valor alojado en $\mathtt{x}$ con el resultado de dicha evaluación.
- El comando **compuesto** $\mathtt{C}_1\,;\mathtt{C}_2$ es la composición secuencial de los comandos $\mathtt{C}_1$ y $\mathtt{C}_2$. Comienza con la ejecución de $\mathtt{C}_1$ en el estado actual del almacenamiento. Si esa ejecución termina, ejecuta $\mathtt{C}_2$ en el estado resultante de $\mathtt{C}_1$. Si la ejecución de $\mathtt{C}_1$ no termina, la ejecución de $\mathtt{C}_1\,;\mathtt{C}_2$ no termina. Es una estructura de control.
- El comando compuesto $\mathbf{\mathtt{if}}\ \mathtt{B}\ \{\mathtt{C}_1\}\ \mathbf{\mathtt{else}}\ \{\mathtt{C}_2\}$ es otra estructura de control. Primero evalúa la expresión booleana $\mathtt{B}$ en el estado actual de almacenamiento; si resulta verdadero, entonces se ejecuta $\mathtt{C}_1$; si B evalúa a falso, se ejecuta $\mathtt{C}_2$.

Semántica del lenguaje de programación (2)

- El comando compuesto **`while`** `B` {`C`} permite ejecutar cosas repetidamente.
  1. La expresión booleana `B` se evalúa en el estado actual del almacenamiento.
  2. Si `B` evalúa a falso, el comando termina.
  3. En caso contrario, el comando `C` se ejecuta. Si esa ejecución termina, se vuelve al paso 1 con una re-evaluación de `B` dado que el estado del almacenamiento pudo haber cambiado su valor.

  La idea del comando `while` es ejecutar `C` repetidamente mientras `B` evalúe a verdadero.

  ¿En qué circunstancias no termina?

Ejemplo: factorial

Listing 1: Ejemplo de código

```
y = 1;
z = 0;
while (z != x) {
   z = z + 1;
   y = y * z
   }
```

- Observemos que nuestro programa calcula el factorial de `x`, ¿seguro?

Especificando, tripletas de Hoare

- Es muy común que nuestras espeficicaciones informales tengan la forma

  "Dado un número positivo x, devuelve un número y tal que $y.y < x$"
- Si se acuerdan, en Programación 1 se reforzaba mucho esta forma de 'especificar' programas informalmente.
- Nuestra idea es formalizar estas especificaciones mediante **Tripletas de Hoare**:

$$(\!|\phi|\!)\ P\ (\!|\psi|\!)$$

- Para nuestro ejemplo, buscamos un programa P tal que:

$$(\!|x > 0|\!)\ P\ (\!|y.y < x|\!)$$

  sea una tripleta válida.

Tripletas de Hoare $(\!|\phi|\!)\ P\ (\!|\psi|\!)$, preguntas

- ¿Qué significa cada parte?
- ¿A qué lenguaje pertenece cada una?
- ¿Qué significado quisiera que tenga que una terna ϕ, P, ψ sea una tripleta válida?
- ¿Qué pasa si ejecuto el programa en un estado donde no se cumple ϕ?
- ¿Para cada par ϕ, ψ hay un P tal que $(\!|\phi|\!)\ P\ (\!|\psi|\!)$ es válida?
- ¿Para cada par ϕ, ψ hay un único P tal que $(\!|\phi|\!)\ P\ (\!|\psi|\!)$ es válida?
- ¿Qué pasa si el programa no termina?
- ¿Quién fue C. A. R. Hoare?

Tripletas de Hoare $(\!|\phi|\!)\ P\ (\!|\psi|\!)$, definiciones

- La fórmula ϕ la llamamos *precondición*
- La fórmula ψ la llamamos *postcondición*
- Ambas fórmulas están definidas sobre una signatura $\mathcal{F} = \{-, \dot{-}, +, *\}$, $\mathcal{P} = \{=, <\}$
- Un *estado* es una función $I : Var \to \mathbb{Z}$
- Decimos que $I \models \phi$ si $[\![\phi]\!]_{\mathcal{M},I} = T$, donde $\mathcal{M}$ es el modelo estándar de los números enteros para $(\mathcal{F}, \mathcal{P})$
- Ejemplo: si I es tal que $I(x) = 2$, $I(y) = -5$, $I(z) = 3$, determinar si:

$$
\begin{aligned}
&I \models 3 * x + y < z \\
&I \models x + y = z \\
&I \models 2 * x - z < y \\
&I \models \exists u(2 * u + x = z) \\
&I \models \forall u(u > 0 \to x * u < z * u)
\end{aligned}
$$

Dos sabores para corrección: parcial y total

Definición (Corrección parcial $\models_{\text{par}}$)

Decimos que una tripleta $(\!|\phi|\!)\ P\ (\!|\psi|\!)$ se cumple bajo corrección parcial ($\models_{\text{par}} (\!|\phi|\!)\ P\ (\!|\psi|\!)$) si, para cualquier estado I que satisface ϕ, si el programa P termina, lo hace en un estado que satisface ψ.

- Si no hay estado inicial que cumpla la precondición, se cumple trivialmente $\models_{\text{par}} (\!|\phi|\!)\ P\ (\!|\psi|\!)$
- Si el programa no termina en ningún estado, se cumple $\models_{\text{par}} (\!|\phi|\!)\ P\ (\!|\psi|\!)$, cualesquiera sean las pre y postcondiciones

Corrección parcial

- Volvamos a la tripleta $(\!|x > 0|\!)\ P\ (\!|y.y < x|\!)$, podemos demostrar que si P es

```
y = 0;
while (y * y < x) {
  y = y + 1
}
y = y - 1
```

  se cumple $\models_{\text{par}} (\!|x > 0|\!)\ P\ (\!|y.y < x|\!)$
- Pero también si P es

  `y = 0`
- Y también, si P es

  `while (true) {y = 1}`

Dos sabores para corrección: parcial y total

Definición (Corrección total $\models_{\text{tot}}$)

Decimos que una tripleta $(\!|\phi|\!)\ P\ (\!|\psi|\!)$ se cumple bajo corrección total ($\models_{\text{tot}} (\!|\phi|\!)\ P\ (\!|\psi|\!)$) si, para cualquier estado I que satisface ϕ, el programa P termina y lo hace en un estado que satisface ψ.

- Si ningún estado inicial cumple la precondición se cumple trivialmente $\models_{\text{tot}} (\!|\phi|\!)\ P\ (\!|\psi|\!)$
- Si el programa no termina para algún estado que cumple la precondición, **no** se cumple $\models_{\text{tot}} (\!|\phi|\!)\ P\ (\!|\psi|\!)$
- Si un estado no cumple la precondición y en ese estado el programa no termina, esto no significa que la tripleta no se cumpla. La obligación de terminar es sólo para los estados que cumplen la pre.

Un cálculo para $\models_{\text{par}}$

$$\frac{(\!|\phi|\!)\ \texttt{C1}\ (\!|\eta|\!) \qquad (\!|\eta|\!)\ \texttt{C2}\ (\!|\psi|\!)}{(\!|\phi|\!)\ \texttt{C1;C2}\ (\!|\psi|\!)}\ \text{Comp}$$

$$\frac{}{(\!|\psi[E/x]|\!)\ \texttt{x=E}\ (\!|\psi|\!)}\ \text{Asig}$$

$$\frac{(\!|\phi \wedge B|\!)\ \texttt{C1}\ (\!|\psi|\!) \qquad (\!|\phi \wedge \neg B|\!)\ \texttt{C2}\ (\!|\psi|\!)}{(\!|\phi|\!)\ \texttt{if B \{C1\} else \{C2\}}\ (\!|\psi|\!)}\ \text{If}$$

$$\frac{(\!|\psi \wedge B|\!)\ \texttt{C}\ (\!|\psi|\!)}{(\!|\psi|\!)\ \texttt{while B \{C\}}\ (\!|\psi \wedge \neg B|\!)}\ \text{WhileP}$$

$$\frac{\vdash_{ARIT} \phi' \rightarrow \phi \qquad (\!|\phi|\!)\ \texttt{C}\ (\!|\psi|\!) \qquad \vdash_{ARIT} \psi \rightarrow \psi'}{(\!|\phi'|\!)\ \texttt{C}\ (\!|\psi'|\!)}\ \text{Debilitamiento}$$

Regla de asignación

Si P es `x = 2`, las siguientes son instancias de la regla:

- $(\!|2 = 2|\!)\ P\ (\!|x = 2|\!)$
- $(\!|2 = 4|\!)\ P\ (\!|x = 4|\!)$
- $(\!|2 = y|\!)\ P\ (\!|x = y|\!)$
- $(\!|2 > 0|\!)\ P\ (\!|x > 0|\!)$

Regla de asignación

Si P es `x = 2`, las siguientes son instancias de la regla:

- $(\!|2 = 2|\!)\ P\ (\!|x = 2|\!)$
- $(\!|2 = 4|\!)\ P\ (\!|x = 4|\!)$
- $(\!|2 = y|\!)\ P\ (\!|x = y|\!)$
- $(\!|2 > 0|\!)\ P\ (\!|x > 0|\!)$

Si P es `x = x+1`, las siguientes son instancias de la regla:

- $(\!|x + 1 = 2|\!)\ P\ (\!|x = 2|\!)$
- $(\!|x + 1 = y|\!)\ P\ (\!|x = y|\!)$
- $(\!|x + 1 + 5 = y|\!)\ P\ (\!|x + 5 = y|\!)$
- $(\!|x + 1 > 0 \wedge y > 0|\!)\ P\ (\!|x > 0 \wedge y > 0|\!)$

Ejercicio resuelto (asignación)

$\models_{\text{par}}((\!|x+1>0 \wedge y>0|\!)\ x=2\ (\!|x>0 \wedge y>0|\!))$

- En árbol usando una herramienta web: `https://proof-tree-builder.github.io/`
- Ingresar el objetivo: `{x+1>0 & y>0} x:=2 {x>0 & y>0}`
- Aplicar cons (debilitamiento/implied)
- Aplicar asgn
- Aplizar z3

$$\dfrac{\dfrac{}{(x+1>0)\wedge(y>0)\vdash(2>0)\wedge(y>0)}\ Z3 \qquad \vdash\{(2>0)\wedge(y>0)\}\ x:=2\ \{(x>0)\wedge(y>0)\}}{\vdash\{(x+1>0)\wedge(y>0)\}\ x:=2\ \{(x>0)\wedge(y>0)\}}\ CONS_*$$

Ver versión *Proof tableaux*.

Ejercicio (asignación)

Resolver en versión *Proof tableaux*: $\models_{\mathrm{par}}((\!|y < 3|\!)\ y = y + 1\ (\!|y < 4|\!))$

$$\dfrac{\dfrac{}{y < 3 \vdash y + 1 < 4}\ Z3 \qquad \vdash \{y + 1 < 4\}\ y := y + 1\ \{y < 4\}}{\vdash \{y < 3\}\ y := y + 1\ \{y < 4\}}\ CONS_*$$

Ejercicio (composición)

$\models_{\mathrm{par}} ((\!| T |\!)\ z = x; z = z + y; u = z\ (\!| u = x + y |\!))$

- En árbol usando una herramienta web: `https://proof-tree-builder.github.io/`
- Ingresar el objetivo: {`true`} `z:=x; z:=z+y; u:=z` {`u=x+y`}
- ...

Ver versión *Proof tableaux*.

Ejercicio (if)

$\models_{\mathrm{par}}((\!| T |\!)\ a = x + 1; \mathit{if}(a - 1 == 0)\{y = 1;\}\mathit{else}\{y = a;\}\ (\!| y = x + 1 |\!))$

- En árbol usando una herramienta web: `https://proof-tree-builder.github.io/`
- Ingresar el objetivo: `{true} a:=x+1; if(a-1=0)then(y:=1)else(y:=a) {y=x+1}`

Ejercicio (if)

$\models_{\text{par}}((\!| T |\!)\ a = x + 1; \mathit{if}(a - 1 == 0)\{y = 1;\}\mathit{else}\{y = a;\}\ (\!| y = x + 1 |\!))$

- En árbol usando una herramienta web: `https://proof-tree-builder.github.io/`
- Ingresar el objetivo: `{true} a:=x+1; if(a-1=0)then(y:=1)else(y:=a) {y=x+1}`

Ver versión *Proof tableaux*.

$$\frac{(\!|\phi_1|\!)\ \texttt{C1}\ (\!|\psi|\!) \quad (\!|\phi_2|\!)\ \texttt{C2}\ (\!|\psi|\!)}{(\!|(B \to \phi_1) \wedge (\neg B \to \phi_2)|\!)\ \texttt{if B \{C1\} else \{C2\}}\ \ (\!|\psi|\!)}\ \text{If2}$$

While

$$\frac{(\!|\eta \wedge B|\!)\ \mathtt{C}\ (\!|\eta|\!)}{(\!|\eta|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\eta \wedge \neg B|\!)}\ \mathrm{WhileP}$$

La fórmula η es un **invariante** del cuerpo C del while: si la guarda B es verdadera y el invariante se cumple antes de ejecutar C y C termina, el invariante sigue valiendo.

Si el while se ejecuta hasta terminar y vale la premisa de WhileP:

- Si B es falsa cuando inicia el while, no se ejecuta C. Nada puede cambiar la verdad de η. Se 'sale' del while con $\eta \wedge \neg$ B.
- Si B es verdadera cuando inicia el while, se ejecuta C (y vale η).
  - Si B se hace falsa, el while termina con $\eta \wedge \neg$ B.
  - Si B permanece verdadera, se ejecuta C nuevamente. El invariante η continúa valiendo (por premisa). Se 'sale' del while únicamente si B es falsa luego de ejecutar finitas veces C, donde tendremos $\eta \wedge \neg$ B.

While (continuación)

$$\frac{(\!|\eta \wedge B|\!)\ \mathtt{C}\ (\!|\eta|\!)}{(\!|\eta|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\eta \wedge \neg B|\!)}\ \mathrm{WhileP}$$

¿Qué pasa si quiero calcular $\models_{\mathrm{par}} (\!|\phi|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\omega|\!)$, para ω distinta de $\phi \wedge \neg \mathtt{B}$?

While (continuación)

$$\frac{(\!|\eta \wedge B|\!)\ \mathtt{C}\ (\!|\eta|\!)}{(\!|\eta|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\eta \wedge \neg B|\!)}\ \text{WhileP}$$

¿Qué pasa si quiero calcular $\models_{\text{par}} (\!|\phi|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\omega|\!)$, para ω distinta de $\phi \wedge \neg \mathtt{B}$?

Debo descubrir un invariante η, tal que:

- $\models_{\text{ARIT}} \phi \rightarrow \eta$,
- $\models_{\text{ARIT}} (\eta \wedge \neg \mathtt{B}) \rightarrow \omega$, y
- $\models_{\text{par}} (\!|\eta|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\eta \wedge \neg B|\!)$.

While (continuación)

$$\frac{(\!|\eta \wedge B|\!)\ \mathtt{C}\ (\!|\eta|\!)}{(\!|\eta|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\eta \wedge \neg B|\!)}\ \text{WhileP}$$

¿Qué pasa si quiero calcular $\models_{\text{par}} (\!|\phi|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\omega|\!)$, para ω distinta de $\phi \wedge \neg \mathtt{B}$?

Debo descubrir un invariante η, tal que:

- $\models_{\text{ARIT}} \phi \rightarrow \eta$,
- $\models_{\text{ARIT}} (\eta \wedge \neg \mathtt{B}) \rightarrow \omega$, y
- $\models_{\text{par}} (\!|\eta|\!)\ \mathtt{while\ B}\ \{\mathtt{C}\}\ (\!|\eta \wedge \neg B|\!)$.

¿Cómo descubrir invariantes?

Ejercicio (while)

Listing 2: Programa Fact

```
y = 1;
z = 0;
while (z != x) {
  z = z + 1;
  y = y * z
  }
```

- Probar $\models_{\text{par}} (\!| x >= 0 |\!)$ Fact $(\!| y = x! |\!)$
- En árbol usando una herramienta web: `https://proof-tree-builder.github.io/`
- Ingresar el objetivo: `{x>0} y:=1; z:=0; while !(z = x) do (z:=z+1; y:=y*z) {y=fact(x)}` ¡Cuidado!